Beyond Encryption: An Integrated Review of Authentication, Protocol Security, and Cryptographic Agility

Sanskar Jadhav , Shumbham Gotad, Dhruv Singh, Ashish Chaudhari
Department of Computer Engineering
Vidyalankar Institute of Technology, Mumbai, India

***Abstract*—Cryptographic systems are often described through the algorithms they use, but an algorithm name alone says little about whether a deployment is safe. A practical design must connect a security goal to an appropriate primitive, use that primitive in a well-defined protocol, and preserve its assumptions through key management and implementation. This paper reviews that chain using foundational research, Internet protocol specifications, and National Institute of Standards and Technology (NIST) publications. It distinguishes encryption, hashing, message authentication, signatures, and key establishment; discusses authenticated encryption and implementation risks; and considers two engineering pressures that shape current design choices: constrained devices and the transition to post-quantum cryptography. A comparison framework is provided to help match cryptographic mechanisms to their intended roles without treating unlike algorithms as competitors on a single scale. The review does not report new performance measurements. Instead, it identifies deployment questions that should be answered before a system is selected, benchmarked, or migrated. The central conclusion is that dependable cryptography is a property of the complete system, including its threat model, protocol composition, operational controls, and capacity to adopt updated standards.**

***Index Terms*—cryptography, authenticated encryption, hash functions, digital signatures, key management, lightweight cryptography, post-quantum cryptography**

I. Introduction

Cryptography protects information in settings where messages may be observed, altered, replayed, or forged. The relevant security requirement depends on the situation. A stored medical record needs confidentiality and integrity; a software update also needs a way to verify who authorized it; and a network connection needs a mechanism for establishing keys and agreeing on the parameters used to protect traffic. Treating all of these needs as simply "encryption" can hide important gaps.

Foundational work established two complementary ways to reason about secrecy. Shannon examined secrecy through information theory, while Diffie and Hellman showed how parties could establish a shared secret over an open channel; the RSA construction further developed practical public-key encryption and signatures [1]–[3]. Modern systems build on these ideas, but their reliability also depends on carefully composed protocols, safe implementations, and the way keys are managed over time.

This paper takes a deployment-oriented view of cryptographic foundations. It makes three contributions. First, it maps common security goals to mechanisms and explains what each mechanism does not provide. Second, it compares standards by function and deployment constraints rather than by an unsupported overall ranking. Third, it draws out practical implications for connected services, constrained devices, and systems preparing for post-quantum algorithms. The goal is not to introduce a new cipher or claim a new benchmark result; it is to clarify how existing foundations inform engineering decisions.

II. Review Scope and Approach

This is a narrative technical review, not a systematic literature review or an experimental evaluation. The source set combines foundational research papers, the TLS 1.3 protocol specification, and NIST standards for encryption, hashing, digital signatures, lightweight cryptography, and post-quantum mechanisms [1]–[8], [10]–[13]. Foundational papers are used to frame the development of the field; standards and protocol specifications are used for standardized names, supported functions, and deployment context.

The comparison uses four qualitative dimensions: (1) the security service provided, (2) the assumptions and failure modes that matter, (3) the operating environment, and (4) lifecycle requirements such as key rotation or algorithm replacement. No implementation was built and no timing, memory, energy, or throughput measurements were taken. Any performance decision therefore requires testing on the target device and workload; the paper does not infer such results from an algorithm's name or design family.

III. Security Goals and Threat Models

A threat model states what an adversary can observe or do. For example, an attacker limited to listening on a network presents a different challenge from one who can modify messages, submit chosen inputs, replay old traffic, or obtain access to a device. A security claim is meaningful only in relation to those capabilities and the assumptions made by the construction.

The principal goals overlap, but they are not interchangeable. Table I summarizes the distinctions that matter during design. In particular, a digest of a message does not prove who produced it: an attacker who changes the message can usually recompute an unkeyed hash. Likewise, a message

TABLE I
SECURITY GOALS AND THE ROLE OF COMMON MECHANISMS

Goal	Typical mechanism	Important boundary
Confidentiality	Encryption; preferably an AEAD construction for data in transit or storage	Does not by itself establish the sender's identity or prevent all misuse of plaintext after decryption.
Integrity	AEAD tag, MAC, or digital signature	A plain hash alone is not sufficient against an attacker who can change the message and recompute the digest.
Entity authentication	Certificate-backed signatures or an authenticated key exchange	Depends on validation, trust configuration, and correct protocol state.
Key establishment	Diffie–Hellman-style exchange or a key-encapsulation mechanism (KEM)	Establishing a secret does not automatically authenticate the parties.
Public verifiability	Digital signature and a trustworthy verification key	A signature does not conceal the signed material.

authentication code (MAC) uses a shared secret, so it can authenticate among parties that hold that secret but does not provide the same public verification property as a digital signature.

Security is consequently better understood as a chain of responsibilities than as one property of one algorithm. Fig. 1 presents that chain at a high level. A weak random-number source, careless nonce handling, or an incorrectly configured certificate check can undermine a system even when it uses a standardized primitive.

IV. Core Cryptographic Mechanisms

A. Symmetric Encryption and Authenticated Encryption

Symmetric cryptography uses secret key material shared by the parties that protect and recover data. The Advanced Encryption Standard (AES) specifies 128-bit blocks and key sizes of 128, 192, or 256 bits [4]. In deployed systems, however, the choice of block cipher is only part of the design. The mode of operation, nonce rules, authentication behavior, and handling of errors are also consequential.

Authenticated encryption with associated data (AEAD) is often a suitable default for application data because it protects confidentiality and verifies integrity in one defined construction. Associated data can bind important fields, such as a protocol header, to the authentication check without encrypting those fields. AES-GCM and ChaCha20-Poly1305 are widely specified examples [4], [8], [9]. Correct nonce handling remains essential: reusing a nonce under the same key can invalidate the intended guarantees for constructions that require nonce uniqueness. Applications should use vetted libraries and follow the specific construction's usage requirements rather than creating a custom combination of encryption and checksums.

Fig. 2 sketches the essential AEAD flow. The receiver must verify the authentication tag before treating the recovered plaintext as valid. If verification fails, the protected data should be rejected; it should not be passed onward as though decryption alone established trust.

B. Hash Functions, MACs, and Signatures

A cryptographic hash function maps an input of arbitrary length to a fixed-length digest. Security analysis commonly considers collision resistance, preimage resistance, and second-preimage resistance. NIST specifies the Secure Hash Standard in FIPS 180-4 and the SHA-3 family in FIPS 202 [5], [6]. A hash is useful for detecting accidental or unauthorized changes when the expected digest is already trusted, but by itself it does not authenticate an untrusted sender.

A MAC addresses that limitation by using a secret key to authenticate a message. Digital signatures use a private signing key and a public verification key, which changes the trust model and allows verification by parties that do not possess the signing secret. FIPS 186-5 specifies signature mechanisms including RSA, ECDSA, and EdDSA [7]. Therefore, these mechanisms should be selected by the required security service, not treated as interchangeable ways to produce a short value next to a message.

C. Public-Key Operations and Key Establishment

Public-key techniques reduced the need for every pair of communicating parties to share a secret in advance. Diffie–Hellman key exchange and later public-key constructions changed how secure communication could be established across open networks [2], [3]. In current systems, asymmetric operations are commonly used for authentication or to establish traffic-key material; symmetric mechanisms then protect the bulk data because they are generally more efficient for that role.

A key exchange should not be confused with identity verification. An unauthenticated key exchange may establish a shared secret with an active attacker rather than the intended peer. Protocols therefore combine key establishment with authentication, transcript binding, and explicit handling of negotiation and state. TLS 1.3 is one standardized example of such protocol composition [8].

V. Protocol Composition and Implementation

A sound primitive can be used in an unsafe protocol. Common design concerns include separating keys by purpose, enforcing nonce requirements, binding handshake messages to the negotiated session, validating certificates, resisting downgrade attempts, and handling errors without disclosing sensitive information. Protocol specifications help define these interactions, but application integration can still introduce failures.

Implementation attacks target behavior that the abstract algorithm description may not reveal. Timing differences, cache behavior, power consumption, fault injection, memory exposure, and unsafe serialization can leak information or

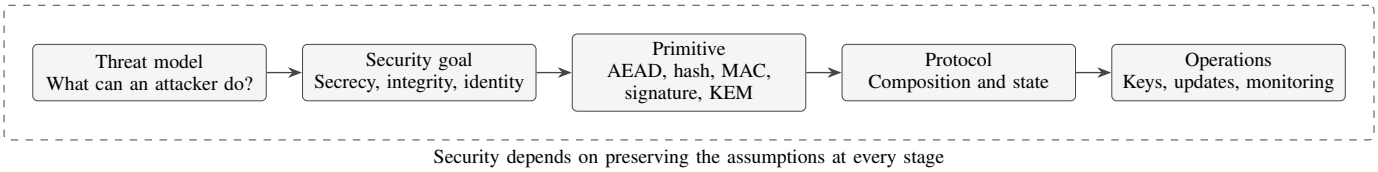


Fig. 1. A deployment-oriented view of cryptographic design. The mechanism is only one stage in a larger security process.

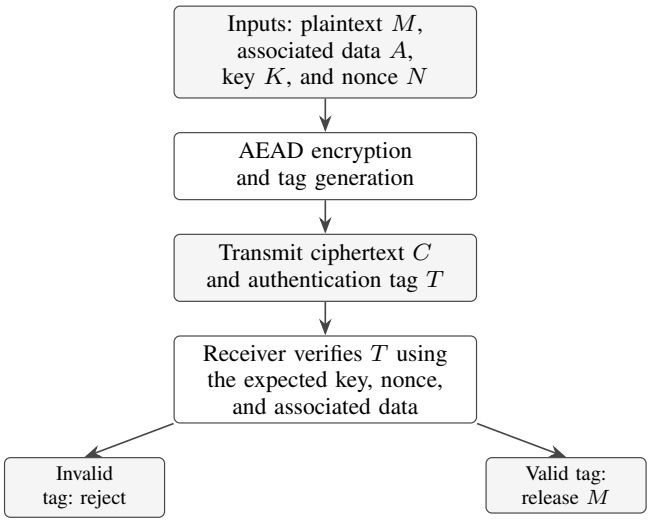


Fig. 2. Simplified AEAD processing. Plaintext is released only after tag verification succeeds.

bypass intended controls. A practical review should therefore consider both mathematical attacks and the device or software environment in which a construction runs. Constant-time implementations can be important for secret-dependent operations, but they are not a substitute for sound key storage, input validation, dependency updates, and correct protocol use.

Key management extends across generation, provisioning, storage, access control, rotation, revocation, backup, and destruction. The right lifecycle depends on the asset: a short-lived session key and a signing key embedded in a device for many years have different operational risks. Inventorying where keys are used and planning how they will be replaced should be treated as part of design rather than an administrative task added at the end.

VI. Cryptography for Constrained Devices

Small sensors and embedded devices may have limited RAM, energy, code space, processing capacity, or hardware acceleration. In those settings, the best design cannot be identified from throughput alone; memory, message overhead, energy per operation, implementation maturity, and the consequences of a failed update also matter.

NIST published SP 800-232 in August 2025, standardizing the Ascon-based functions Ascon-AEAD128, Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128 for constrained environments [10]. This gives designers a standardized option to consider when the target platform and application are a good fit. It does not mean Ascon is automatically preferable in every device: AES may be more appropriate where hardware acceleration, existing validation, interoperability, or library support make it the safer operational choice.

For a real deployment, a useful comparison should be run on the intended hardware with representative message sizes, key lifetimes, and error conditions. The measurements should include memory and energy where those constraints drive the design, and should be paired with a review of nonce requirements, update mechanisms, and cryptographic library support. This review has not performed those measurements, so it makes no numerical performance claim about Ascon or AES.

VII. Post-Quantum Cryptography and Migration

Large-scale quantum computing could change the security of widely deployed public-key schemes whose security depends on factoring or discrete logarithms. The impact is not identical across all cryptographic mechanisms: the most immediate migration planning concerns public-key key establishment and signatures, while symmetric cryptography and hash functions require a different analysis.

In August 2024, NIST published three post-quantum standards: FIPS 203 for the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), FIPS 204 for the Module-Lattice-Based Digital Signature Algorithm (ML-DSA), and FIPS 205 for the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA) [11]–[13]. These standards provide concrete algorithms for systems that need to begin compatibility and migration work. The transition is not simply a matter of swapping an algorithm name: libraries, protocols, certificates, message sizes, device constraints, and trust relationships can all be affected.

NIST selected HQC for standardization in March 2025 as an additional key-encapsulation mechanism, but selection for standardization should not be confused with publication of a finalized standard [14]. Implementers should check the current NIST project status before planning production adoption. NIST has also posted a note that FIPS 203 may receive a future update or correction, which is a reminder to consult official errata and current guidance during implementation [11].

A migration plan should begin with a cryptographic inventory: identify where RSA, elliptic-curve methods, certificates, signatures, and key establishment are used; determine which assets must remain confidential or verifiable for many years; and map dependencies across devices and services. The next steps are to test supported implementations and protocol

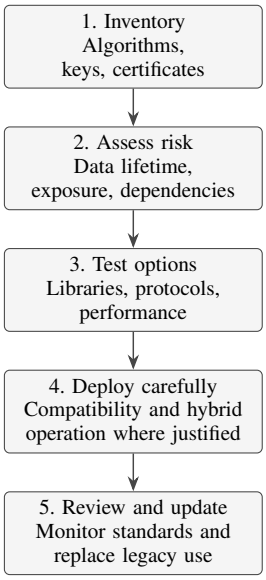


Fig. 3. A staged migration path for post-quantum readiness. Actual sequencing depends on system risk and compatibility constraints.

compatibility, decide where a hybrid design is justified, and introduce algorithm agility so that future changes do not require a complete redesign. Fig. 3 summarizes this staged approach.

VIII. Comparative Analysis and Design Implications

No single primitive is best for every use case because the mechanisms provide different services and carry different operating assumptions. Table II offers a role-based comparison, not a performance ranking. It is intended as an initial decision aid; a production design still needs a threat model, implementation review, interoperability testing, and current standards guidance.

Three deployment examples illustrate how the comparison can be used. For a web API, the main task is not to invent a new encryption layer; it is to configure a maintained TLS implementation, validate certificates, protect server keys, and ensure the application does not mishandle authenticated data. For a sensor with strict energy or memory limits, Ascon is worth evaluating against the actual workload, but any choice should be supported by measurements and a realistic update strategy. For a long-lived service that relies on public-key signatures or key establishment, the immediate work is inventory and compatibility testing: a migration that ignores certificate formats, device replacement cycles, or protocol dependencies can fail operationally even when the new primitive is sound.

These examples lead to a broader point. A standards document defines an algorithm or protocol precisely, but it cannot decide whether a particular deployment has correct key permissions, safe defaults, monitoring, and rollback procedures. Those choices belong to the system design and operations around the standard. The engineering comparison should therefore keep cryptographic strength, performance, deployability, and lifecycle resilience as separate questions rather than collapsing them into one score.

TABLE II
Role-based comparison of selected mechanisms

Mechanism	Role	Decision note
AES in AEAD mode	Confidentiality and integrity	Follow mode-specific nonce and key-use rules.
SHA-2 / SHA-3	Message digests and construction components	An unkeyed hash alone does not authenticate an untrusted sender.
MAC	Shared-secret message authentication	Any holder of the shared key may be able to create a valid tag.
FIPS 186-5 signatures	Digital signing and public verification	Protect the signing key; signatures do not hide the message.
TLS 1.3	Secure transport	Certificate validation and application integration still matter.
Ascon (SP 800-232)	Lightweight cryptography	Benchmark on the target device; do not assume it is always faster.
ML-KEM / ML-DSA / SLH-DSA	Post-quantum key establishment and signatures	Check compatibility, current revisions, and migration dependencies.

IX. Limitations

This paper summarizes selected foundational publications and standards; it does not claim exhaustive coverage of cryptography or a systematic search of every relevant publication. The comparison is qualitative and no benchmark or formal security proof is provided. Performance depends on hardware, libraries, configuration, message patterns, and implementation details, so the paper intentionally avoids numerical speed or energy claims. Standards and implementation guidance can also change. Before deployment, readers should verify current NIST publications, errata, protocol guidance, and the security advisories for their chosen libraries.

X. Conclusion

Cryptographic foundations are most useful when they are connected to a concrete threat model and the actual life of a system. Encryption, hashing, MACs, signatures, and key establishment solve different problems; protocol composition determines how their guarantees work together; and key management and implementation practice determine whether those guarantees survive deployment. Lightweight standards such as Ascon provide an option for constrained environments, while ML-KEM, ML-DSA, and SLH-DSA create a basis for post-quantum migration planning. Neither development removes the need for system-specific evaluation. A responsible design documents why a mechanism fits its role, tests it in the target environment, tracks changes to standards, and plans for replacing keys and algorithms before a crisis makes that replacement urgent.

ACKNOWLEDGMENT

The author acknowledges the CYCRAI track materials that motivated the review scope and the IEEE conference template used for formatting.

REFERENCES

[1] C. E. Shannon, "Communication theory of secrecy systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, Oct. 1949, doi: 10.1002/j.1538-7305.1949.tb00928.x.

[2] W. Diffie and M. E. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, Nov. 1976, doi: 10.1109/TIT.1976.1055638.

[3] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978, doi: 10.1145/359340.359342.

[4] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197-upd1, May 2023, doi: 10.6028/NIST.FIPS.197-upd1.

[5] Q. H. Dang, *Secure Hash Standard*, NIST FIPS PUB 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.

[6] National Institute of Standards and Technology, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, FIPS PUB 202, Aug. 2015, doi: 10.6028/NIST.FIPS.202.

[7] L. Chen, D. Moody, A. Regenscheid, and A. Robinson, *Digital Signature Standard (DSS)*, NIST FIPS PUB 186-5, Feb. 2023, doi: 10.6028/NIST.FIPS.186-5.

[8] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force, Aug. 2018, doi: 10.17487/RFC8446.

[9] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, Internet Engineering Task Force, June 2018, doi: 10.17487/RFC8439.

[10] M. Sönmez Turan, K. McKay, J. Kang, J. Kelsey, and D. Chang, *Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions*, NIST SP 800-232, Aug. 2025, doi: 10.6028/NIST.SP.800-232.

[11] National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS PUB 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203.

[12] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS PUB 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204.

[13] National Institute of Standards and Technology, *Stateless Hash-Based Digital Signature Standard*, FIPS PUB 205, Aug. 2024, doi: 10.6028/NIST.FIPS.205.

[14] National Institute of Standards and Technology, "HQC announced as a 4th round selection," Post-Quantum Cryptography Standardization Process, Mar. 11, 2025. [Online]. Available: https://csrc.nist.gov/News/2025/hqc-announced-as-a-4th-round-selection